

پروژه دوم آزمایشگاه سیستم عامل

مجید سپاهی - 810100163

امیرحسین صباحی - 810101556

پرسش 1: کتابخانه های سطح کاربر، موجود در دایرکتوری ULIB (مانند فایل های `usys.S` و `ulib.c`) توابع پوشاننده های را ارائه میدهند که این فراخوانی های سیستمی را به صورت انتزاعی مدیریت میکنند. توضیح دهید که این کتابخانه ها چگونه با استفاده از ماکروها و توابع پوشاننده، جزئیات فراخوانی های سیستمی (مانند شماره فراخوانی، آرگومانها و بازگردانی مقادیر) را از برنامه نویس پنهان میکنند. همچنین، دلایل استفاده از این انتزاع در بهبود قابلیت حمل، افزایش امنیت و ساده سازی توسعه برنامه های کاربر را بیان نمایید.

در `xv6` برای برنامه های کاربری مجموعه ای از “کتابخانه های سطح کاربر” وجود دارد که زیرساخت لازم برای صدا زدن فراخوانی های سیستمی را فراهم می کنند و در عین حال جزئیات سخت افزاری/سیستمی را از دید برنامه نویس پنهان می سازند. مهم ترین اجزای این لایه عبارت اند از فایل اسمبلی `usys.S`، فایل های هدر مانند `traps.h` و `syscall.h` و اعلان توابع در `user.h`، و همچنین پیاده سازی برخی توابع کمکی در `ulib.c` که روی فراخوانی های سیستمی سوار می شوند. ایده اصلی این است که برنامه کاربر به جای آنکه خودش شماره فراخوانی سیستم، نحوه بارگذاری آرگومانها در ثباتها یا فراخوانی وقفه نرم افزاری را بداند، فقط یک تابع `C` معمولی مثل `write` یا `fork` را صدا بزند؛ بقیه کار را ماکروها و توابع پوشاننده انجام می دهند.

مکانیزم پنهان سازی جزئیات با ماکروها در `usys.S`

فایل `usys.S` با استفاده از پیش پردازنده `C (cpp)` و ماکروها، برای هر فراخوانی سیستمی یک “دروازه کوچک (stub)” می سازد. این استابها توابعی هستند که از دید برنامه نویس مثل یک تابع `C` معمولی به نظر می رسند، اما وقتی اجرا می شوند چند کار مشخص انجام می دهند: شماره فراخوانی سیستم را در ثبات `eax` قرار می دهند، وقفه مخصوص فراخوانی سیستم را فراخوانی می کنند (در `xv6-x86` وقفه نرم افزاری با شماره ثابت `T_SYSCALL`، و سپس مقدار بازگشتی را که هسته در `eax` قرار داده تحویل می دهند.

```

\ #define SYSCALL(name) \
\ .globl name; \
\ name: \
\     movl $SYS_ ## name, %eax; \
\     int $T_SYSCALL; \
\     ret

```

با استفاده از این ماکرو برای هر نام فراخوانی سیستمی مثلاً `open`، `fork`، `write` و یک برچسب سراسری (نام تابع) تعریف می‌شود و سه دستور اسمبلی تولید می‌گردد: ۱) گذاشتن شماره سیستم‌کال مربوطه در `eax` با استفاده از ثابت‌های `SYS_name` که در `syscall.h` تعریف شده‌اند، ۲) اجرای وقفه `T_SYSCALL` که در `traps.h` تعریف شده، ۳) بازگشت از تابع. مزیت بزرگ این الگو این است که تمام استاب‌ها یکنواخت و خیلی مختصر تولید می‌شوند و لازم نیست برای هر فراخوانی فایل اسمبلی جداگانه‌ای بنویسیم.

همگامی اعلان‌ها و شماره‌ها با هدرها

شماره هر فراخوانی سیستم در فایل هدر `syscall.h` تعریف می‌شود مانند `SYS_fork`، `SYS_write` و... همچنین شماره وقفه مخصوص فراخوانی سیستم (مثلاً `T_SYSCALL` در `traps.h` می‌آید. اعلان توابع کاربری متناظر (prototype) نیز در `user.h` قرار دارد تا کامپایلر C بتواند صحت تعداد و نوع آرگومان‌ها را بررسی کند. این سه بخش با هم تضمین می‌کنند که:

- ۱) نام تابعی که برنامه‌نویس می‌بیند با استابی که واقعاً اجرا می‌شود یکی باشد،
- ۲) شماره صحیح فراخوانی به هسته ارسال شود،
- ۳) قرارداد فراخوانی (تعداد و نوع آرگومان‌ها، نوع مقدار بازگشتی) برای کامپایلر روشن باشد.

چگونگی عبور آرگومان‌ها و بازگشت نتیجه

وقتی برنامه‌نویس تابعی مانند `write(fd, buf, n)` را صدا می‌زند، طبق قرارداد فراخوانی زبان C روی x86، آرگومان‌ها روی پشته کاربر قرار می‌گیرند و فراخوانی تابع `usys.S` انجام می‌شود. استاب فقط شماره سیستم‌کال را در `eax` می‌گذارد و وقفه را صدا می‌زند؛ هسته xv6 در سمت مقابل، هنگام رسیدن وقفه، رجیسترها و تصویر پشته فرایند را دارد، از روی چارچوب پشته آرگومان‌ها را استخراج می‌کند، آن‌ها را بررسی و اعتبارسنجی می‌کند (مثلاً اینکه اشاره‌گر `buf` واقعاً به حافظه کاربر و قابل دسترسی اشاره می‌کند (و سپس تابع هسته‌ای متناظر را اجرا می‌کند. خروجی (موفقیت یا کد خطا) در `eax` برگردانده می‌شود، کنترل به استاب برمی‌گردد و استاب مستقیماً `return` می‌کند؛ بنابراین برنامه‌نویس در C همان مقدار را به عنوان مقدار بازگشتی تابع می‌بیند. این روند باعث می‌شود که از دید برنامه کاربر، فراخوانی سیستم هیچ تفاوتی با یک تابع کتابخانه‌ای معمولی نداشته باشد.

نقش **ulib.c** و توابع پوشاننده سطح بالاتر

فایل **ulib.c** مجموعه‌ای از توابع کتابخانه‌ای سطح کاربر را پیاده‌سازی می‌کند: توابع ساده رشته‌ای و حافظه‌ای مانند **strcpy**، **strcmp**، **strlen**، **memset** و همچنین توابعی مانند **printf**، **gets** و **malloc/free**. نکته مهم این است که بسیاری از این توابع سطح بالاتر خودشان روی فراخوانی‌های سیستمی تکیه می‌کنند. برای مثال:

printf • برای چاپ، در نهایت **write** را صدا می‌زند و قالب‌بندی را در فضای کاربر انجام می‌دهد.
gets • برای خواندن از ورودی استاندارد از **read** استفاده می‌کند.
malloc • برای گرفتن حافظه بیشتر از هسته معمولاً با ساب‌سیستم مدیریت حافظه در فضای کاربر کار می‌کند و وقتی نیاز به گسترش **heap** باشد از **sbrk** (یک سیستم کال) کمک می‌گیرد.
به این ترتیب، برنامه‌نویس بیشترِ زمانش با توابع آشنا و ساده کتابخانه‌ای سر و کار دارد و فقط در جاهایی که لازم است مستقیم توابعی مانند **open**، **read**، **write**، **fork**، **exec** و ... را فراخوانی می‌کند؛ در هر دو حالت، جزئیات سخت‌افزاری/هسته‌ای پنهان شده‌اند.

• **printf.c**

در این فایل توابع **printf**، **printint**، **putc** وجود دارند که تنها تابعی که در این بخش از فراخوانی سیستمی استفاده می‌کند، تابع **putc** است.
در تابع **putc** از فراخوانی سیستمی **write** به منظور چاپ کردن استفاده شده است. که برای آن **fd** مورد نظر جهت چاپ کردن و کاراکتر مورد نظر انتخاب شده است.

• **umalloc.c**

در این فایل توابع **malloc** و **free** و **morecore** تعریف شده‌اند که **malloc** و **free** کار تخصیص و آزاد کردن حافظه را به عهده دارند و در آنها از فراخوانی‌های سیستمی استفاده نشده است.
در تابع **morecore** از فراخوانی سیستمی **sbrk** استفاده شده است که این فراخوانی سیستمی، حافظه پرداز را گسترش می‌دهد.

قابلیت حمل

اگر پلتفرم یا معماری تغییر کند، یا مکانیسم فراخوانی سیستم عوض شود، تنها لازم است استاب‌ها و هدرهای متناظر را با شرایط جدید تطبیق دهیم. مثال کلاسیک همین **xv6** است که برای معماری‌های دیگر (مثل **RISC-V**) از دستور **ecall** و قراردادهای متفاوتی برای رجیسترها و شماره‌ها استفاده می‌شود. اما برنامه کاربر که تابعی به نام **write** یا **fork** را صدا می‌زند، نیاز به تغییر ندارد و با یک باز کامپایل روی کتابخانه سطح کاربر جدید، روی معماری مقصد نیز کار خواهد کرد. این یعنی **coupling** بین برنامه کاربر و سازوکار واقعی سیستم کال به حداقل می‌رسد و قابلیت حمل به‌طور چشمگیری بهبود می‌یابد.

امنیت

استاب‌ها و لایه کتابخانه‌ای تضمین می‌کنند که ورود از فضای کاربر به فضای هسته فقط از یک دروازه کنترل‌شده اتفاق بیفتد. برنامه کاربر مستقیماً با شماره وقفه‌ها یا سطح دسترسی سخت‌افزاری سروکار ندارد؛ در عوض، هسته در نقطه یکتایی تمام آرگومان‌ها را بررسی می‌کند، اشاره‌گرها را اعتبارسنجی می‌کند، اندازه‌ها و مرزها را می‌سنجد و در صورت بروز خطا مقدار بازگشتی مناسبی می‌دهد. این طراحی از سوءاستفاده‌های احتمالی می‌کاهد: کاربر نمی‌تواند هر وقفه‌ای را دل‌خواهی صدا بزند، رجیسترهای حساس را دستکاری کند یا مستقیماً به آدرس‌های کرنل دسترسی بگیرد. حتی اگر برنامه کاربر اشتباه کند (مثلاً اشاره‌گر نامعتبر بدهد)، کنترل خطا در سطح هسته انجام می‌شود و از خراب‌کردن حالت کرنل جلوگیری می‌شود.

ساده‌سازی توسعه

نوشتن برنامه‌های کاربر با API های شبیه استاندارد C بسیار ساده‌تر از دست‌وپنجه نرم کردن با ثبات‌ها و وقفه‌هاست. برنامه‌نویس لازم نیست بداند شماره write چند است، یا کدام ثبات باید بارگذاری شود؛ کافی است prototype تابع را ببیند و مانند هر تابع C دیگری صدا بزند. مدیریت الگوی خطا هم یک‌دست می‌شود: در xv6 معمولاً بازگشت (-1 نشانگر خطاست) در مقابل errno در یونیکس‌های کامل‌تر)، و برنامه‌نویس به‌روشنی می‌تواند خطا را کنترل کند. این یکنواختی توسعه را سریع‌تر، کد را خواناتر و اشکال‌زدایی را آسان‌تر می‌کند. همچنین توابع پوشاننده سطح بالاتر مانند printf یا gets اجازه می‌دهند که منطق ورودی/خروجی و قالب‌بندی در فضای کاربر انجام شود و فقط عملیات هسته‌ای ضروری (خواندن/نوشتن) به سیستم‌کال‌ها سپرده شود؛ این کار هم کارایی را بهبود می‌دهد (چون از سویچ زمینه بی‌مورد اجتناب می‌شود) و هم پیچیدگی را از دوش هسته برمی‌دارد.

هماهنگی و نگهداری ساده‌تر

تعریف متمرکز شماره‌های سیستم‌کال در syscall.h، اعلان یکنواخت توابع در user.h، و تولید خودکار استاب‌ها با ماکرو در usys.S باعث می‌شود هر تغییری در API سیستم‌کال‌ها فقط در چند نقطه شناخته‌شده اعمال شود. اگر یک سیستم‌کال جدید اضافه کنیم، کافی است یک شماره جدید در syscall.h بگذاریم، یک prototype در user.h اضافه کنیم و یک خط SYSCALL(name) به usys.S اضافه کنیم. این نظم، هزینه نگهداری را کم و احتمال خطاهای ناسازگاری را کاهش می‌دهد.

حداقلی و پرکارایی بودن استاب‌ها

استاب‌های تولیدشده بسیار نازک و کم‌هزینه‌اند: تنظیم یک ثبات، یک وقفه نرم‌افزاری، و بازگشت. این یعنی سربار اضافی سرراست و قابل‌پیش‌بینی. هرگونه کار پیچیده‌تر (مانند قالب‌بندی چاپ یا مدیریت حافظه کاربر) یا

در فضای کاربر انجام می‌شود (ulib.c) یا در هسته در نقطه مناسب. بنابراین این لایه انتزاعی ضمن پنهان‌سازی جزئیات، هزینه قابل توجهی به مسیر بحرانی اضافه نمی‌کند.

پرسش 2: مقایسه ای بین دستور int و دستورات جدید مانند sysenter یا sysexit در دو سیستم عامل مختلف برای مثال Linux و xv6 انجام دهید. در این مقایسه به موارد زیر توجه کنید:

1. نحوه پیاده سازی و عملکرد این دستورات: چگونه هر کدام از این دستورات به فراخوانی سیستم در سطح هسته پرداخته و از چه مکانیزم هایی استفاده میکنند؟
2. کارایی: چگونه دستور int در مقایسه با دستورات جدیدتر مانند sysenter و sysexit از نظر عملکرد و سربار پردازشی در سیستم های عامل مختلف عمل میکند؟
3. کاربردهای مختلف: در چه شرایطی یکی از این دستورات نسبت به دیگری برتری دارد و چرا؟

مقایسه int با sysenter/sysexit در Linux و xv6

1. نحوه پیاده سازی و عملکرد

در xv6 فراخوانی سیستم با `int T_SYSCALL` انجام میشود `cpu`. با درگاه وقفه وارد کرنل میشود، پردازنده `eip/eflags/esp` را روی استک کرنل میگذارد، هندلر شماره سیستم کال را خوانده و در پایان با `iret` برمیگردد.

در Linux 32bit قدیمی: هم `int 0x80` و هم مسیر تندتر `sysenter/ sysexit` وجود دارد .
`sysenter` با MSR ها (`SYSENTER_CS/EIP/ESP`) نقطه ورود سریع را تنظیم میکند، با حداقل ذخیره وضعیت وارد کرنل میشود و با `sysexit` برمیگردد.

در Linux 64bit: عموماً از دستور `syscall/sysret` استفاده میشود. جایگزین `sysenter` در xv6 `x86_64` آموزشی است و فقط `int` را به کار میگیرد.

2. کارایی

`int`: سربار بیشتر به خاطر بررسی درگاه وقفه و ذخیره خودکار چند رجستر. کندتر.
`sysenter/sysexit`: ورود و خروج کم هزینه تر، تاخیر کمتر برای سیستم کال های پرتکرار. در Linux مدرن، `sysenter` یا بهتر از آن `syscall` روی `x86_64` معمولاً سریع تر از `int` است. در xv6 به خاطر سادگی همان `int` استفاده میشود و بهینه سازی کارایی هدف نیست.

3. کاربرد

وقتی سازگاری و سادگی مهم است یا سخت افزار کهنه/ماشین مجازی داریم `int` مناسب تر است هم در xv6 و هم برای مسیر سازگاری در Linux.

برای کارایی بالا در پردازنده های نو: در 32 بیت `sysenter/sysexit` بهتر است؛ در 64 بیت دستور `syscall` برتر است.

در سیستم های آموزشی مثل `xv6` که شفافیت مهم است `int` ساده تر و قابل فهم تر است، هرچند کندتر.

پرسش 3: با توجه به توضیحات ارائه شده درباره ی `Gate Descriptor` ها در `xv6` و نحوه ی تنظیم سطح دسترسی برای فراخوانی های سیستمی، چرا تنها فراخوانی سیستمی (که با `Trap Gate` پیاده سازی شده) با سطح دسترسی `DPL_USER` فعال میشود و سایر تله ها (مانند وقفه های سختافزاری و استثناها) نمیتوانند از این سطح دسترسی بهره ببرند؟

در `x86` هر ورودی از جدول `IDT` یک `gate descriptor` دارد که نوع (`trap/interrupt`)، سگمنت کرنل، و `DPL` را مشخص میکند. تعیین میکند آیا کد با سطح دسترسی کمتر مثل `user` با `CPL=3` مجاز است با دستور نرم افزاری `int` مستقیماً وارد آن گیت بشود یا نه. ایده کلی در `xv6` این است که فقط یک ورودی ویژه برای `syscall` را عمداً برای `user` قابل فراخوانی کنیم و بقیه مسیرها را بسته نگه داریم.

1. کنترل ورودی مجاز از `user` به کرنل

فقط گیت مربوط به سیستم کال بردار `T_SYSCALL` با `DPL_USER` تنظیم میشود تا کد `user` بتواند با `T_SYSCALL` وارد کرنل شود. این ورودی قرارداد روشنی دارد: شماره سیستم کال در رجستر، شکل آرگومانها، سوئیچ معتبر به استک کرنل و کدهای خطای قابل پیشبینی. اگر بقیه بردارها هم `DPL=3` بودند، کد `user` میتواند به هر هندلر داخلی کرنل با `int` سر بزند و از قرارداد درست عبور کند که خطرناک است.

2. وقفه های سخت افزاری و استثناها نباید قابل فراخوانی توسط `user` باشند

این رویدادها یا از بیرون `CPU` میآیند `IRQ` ها یا توسط خود `CPU` تولید میشوند مثلاً `page fault`، `divide error`. `CPU` مستقل از `DPL` گیت، اینها را به کرنل تحویل میدهد. اما اگر `user` بتواند همان بردارها را با `int` صدا بزند، میتواند هندلرهایی را که برای بافت خطای خاص و حالت سخت افزاری خاص طراحی شدهاند، در زمان و بافت دلخواه اجرا کند:

– امکان دور زدن اعتبارسنجی ورودی و عبور از مسیرهای خطای ظریف
– تحریک هندلرهای `IRQ` و جعل رویداد سخت افزاری

وارد کردن کرنل به مسیرهای استثنا با state ناقص یا ارور کد اشتباه
برای جلوگیری، DPL این بردارها صفر می ماند تا هر تلاش از int به user به GPF منجر شود.
3. تفاوت interrupt gate و trap gate

xv6 برای سیستم کال از trap gate استفاده میکند تا بیت وقفه ها (IF) حفظ شود و زمانبندی/تعامل با وقفه ها طبق نیاز سیستم باقی بماند. برای IRQ ها عموماً interrupt gate گذاشته میشود تا هنگام ورود وقفه ها به کرنل، IF پاک شود و تو در تو شدن ناخواسته وقفه ها کنترل گردد. در هر دو حالت، داشتن $DPL=0$ برای غیر sys کال باعث میشود تنها منبع مشروع ورود، خود CPU یا کنترلر وقفه باشد نه user.
4. نتیجه امنیتی و طراحی

با باز گذاشتن فقط یک دروازه با DPL_USER ، کرنل یک نقطه ورود مشخص، قابل حسابرسی و محافظت شده برای تعامل user دارد. همه فراخوانیها از همان مسیر عبور میکنند، اعتبارسنجی آرگومانها یکپارچه است و سطح حمله کوچک میشود. در مقابل، بستن بقیه بردارها برای user جلوی جعل وقفه و سوءاستفاده از هندلرهای داخلی را میگیرد.

پرسش 4: در صورت تغییر سطح دسترسی، ss و esp روی پشته Push میشود. در غیراینصورت Push نمیشود. چرا؟

- چون وقتی سطح دسترسی عوض میشود پردازنده باید به استک امن کرنل سوئیچ کند، ولی وقتی سطح عوض نمیشود نیازی به سوئیچ و ذخیره اطلاعات مربوط به استک قبلی نیست.
1. Cpu هنگام ورود از سطح کاربر به کرنل مثلاً از $CPL=3$ به $CPL=0$ طبق TSS مقادیر $SS0$ و $ESP0$ را بارگذاری میکند و به استک کرنل میپرد. برای اینکه بعداً با `iret` بتواند به همان استک کاربر برگردد، باید زوج $SS:ESP$ قدیمی را هم جایی نگه دارد؛ پس SS و ESP کاربر را روی استک جدید کرنل Push میکند. بدون این اطلاعات بازگشت امن و درست به سطح کاربر ممکن نیست.
 2. اگر ورود در همان سطح رخ دهد مثلاً یک وقفه یا تله حین اجرای کد کرنل در $CPL=0$ ، پردازنده روی همان استک فعلی می ماند؛ SS تغییر نمیکند و ESP هم معتبر است. پس ذخیره SS و ESP اضافه و غیرضروری است و قالب فریم بازگشت `iret` هم در این حالت چنین انتظاری ندارد. در این حالت فقط EIP ، CS ، $EFLAGS$ و در صورت وجود کد خطا Push میشوند.
- SS و ESP فقط وقتی Push میشوند که گذر از مرز سطوح رخ دهد تا سوئیچ استک انجام شود و امکان بازگشت به استک و سطح قبلی فراهم بماند؛ در غیر این صورت همان استک فعلی استفاده میشود و نیازی به ذخیره $SS:ESP$ نیست.

پرسش 5: با توجه به توضیحاتی که در مورد نحوه قرارگیری پارامترهای فراخوانی سیستمی بر روی پشته و نحوه دسترسی به آنها از طریق توابعی مانند `argint` و `argptr` داده شده است، توضیح دهید که این توابع چه نقشی در بازیابی پارامترهای فراخوانی سیستمی دارند. به خصوص، چرا تابع `argptr` باید بازه های آدرس ورودی را بررسی کند؟ در صورت عدم انجام این بررسی ها، چه نوع مشکلات امنیتی مانند دسترسی به حافظه خارج از محدوده مجاز ممکن است ایجاد شود؟ به عنوان مثال، شرح دهید که چگونه نبود این بررسی ها در فراخوانی سیستمی `read_sys` میتواند باعث بروز اختلال یا آسیب پذیری در سیستم شود. همچنین عنوان کنید که آیا حذف چک کردن بازه آدرس عملی و قابل پیاده سازی است یا خیر؟ یا آیا میتوان عملی بازه اشتباه وارد کنیم؟

در `xv6` آرگومانهای سیستم کال روی پشته فرایند کاربر قرار میگیرد و کرنل برای بازیابی آنها از توابع کمکی استفاده میکند. هدف این توابع این است که کرنل بدون اعتماد به داده های کاربر بتواند مقدارها و نشانی ها را به شکل ایمن و قابل اطمینان دریافت کند.

تابع `argint` برای خواندن یک آرگومان عددی به کار میرود. این تابع محل درست آرگومان را نسبت به چارچوب پشته کاربر پیدا میکند مقدار را از حافظه کاربر میخواند و آن را برای مسیر سیستم کال آماده میکند به این ترتیب کرنل مجبور نیست ساختار پشته کاربر را حدس بزند و خطر خواندن از محل نادرست کم میشود.

تابع `argptr` برای آرگومانهای از نوع اشاره گر به کار میرود. این تابع تنها نشانی را تحویل نمیدهد بلکه بازه آدرسی را که قرار است خوانده یا نوشته شود اعتبارسنجی میکند. اعتبارسنجی یعنی اطمینان از اینکه آغاز و پایان بازه داخل فضای آدرس معتبر فرایند کاربر قرار دارد صفحه ها وجود دارند و جمع نشانی و اندازه باعث سرریز عددی و برگشت به ابتدای فضا نمیشود. این کار جلوی دسترسی ناخواسته به حافظه کرنل یا به بخشهای خارج از مالکیت فرایند را میگیرد.

دلیل اصلی لزوم این بررسی در `argptr` این است که اشاره گرهای دریافتی از برنامه کاربر قابل اعتماد نیستند برنامه کاربر میتواند به عمد یا بر اثر خطا نشانی نزدیک مرز فضای خودش را بدهد و اندازه بزرگ انتخاب کند تا نوشتن یا خواندن از مرز عبور کند. اگر کرنل بدون چک ادامه دهد ممکن است به صفحه ناموجود دسترسی پیدا کند و باعث خطای صفحه در حالت کرنل شود یا حتی به حافظه کرنل دست بزند و ساختارهای درونی را خراب کند که زمینه ارتقای امتیاز و نفوذ را فراهم میکند. همچنین اگر کرنل بدون اعتبارسنجی از حافظه کاربر بخواند

ممکن است بیش از حد بخواند و داده های تصادفی یا حساس را ناخواسته افشا کند، در مسیر فراخوانی خواندن نمونه روشنی وجود دارد.

سیستم کال خواندن ابتدا عدد مربوط به توصیفگر فایل را میگیرد سپس با `argptr` بافر کاربر و اندازه را دریافت و اعتبارسنجی میکند و بعد داده ها را در همان بافر میریزد. اگر اعتبارسنجی حذف شود کاربر میتواند با دادن نشانی نامعتبر باعث شود کرنل در جایی خارج از محدوده مجاز بنویسد یا به یک صفحه ناموجود دسترسی پیدا کند و مسیر کرنل را به خرابی بکشانند. اگر نشانی به بخشی از حافظه کرنل اشاره کند نوشتن در آن میتواند به تغییر ناخواسته وضعیت کرنل و حتی اجرای کد مخرب منجر شود.

در کنار `argint` و `argptr` تابع `argstr` هم برای آرگومانهای رشته ای وجود دارد که مانند `argptr` عمل میکند با این تفاوت که علاوه بر بازه باید وجود کاراکتر پایان رشته و قرار داشتن کل رشته در محدوده معتبر را هم چک کند، این نیز جلوگیری میکند از اینکه کرنل تا بی نهایت یا تا خارج از محدوده پیش برود.

در طراحی سیستم عامل های واقعی حذف چک بازه عملی و ایمن نیست، مسیرهای کپی از کاربر به کرنل و برعکس همیشه باید اعتبارسنجی انجام دهند. در غیر این صورت راهی مستقیم برای از کار انداختن سیستم افشای داده و ارتقای امتیاز باز میشود. کاربر البته میتواند بازه اشتباه ارسال کند اما کرنل سالم باید آن را تشخیص دهد و فرایند را متوقف کند یا خطا برگرداند و هرگز نباید عملیات حافظه را با نشانی نامعتبر ادامه دهد.

در فراخوان سیستمی، `sys_read` آرگومانها به صورت زیر دریافت و بررسی میشوند:

```
69 int
70 sys_read(void)
71 {
72     struct file *f;
73     int n;
74     char *p;
75
76     if(argfd(0, 0, &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
77         return -1;
78     return fileread(f, p, n);
79 }
80
```

اگر `argptr` بازه آدرس رو چک نکند، کاربر میتونه نشانی `buf` رو نزدیک آخر حافظه خودش بدهد و یک `n` بزرگ انتخاب کند تا نوشتن کرنل از محدوده مجاز خارج بشود و سیستم به خطای صفحه بخورد و به هم بریزد. یا میتواند `buf` را به نشانی بالای `kernbase` بدهد تا کرنل به جای حافظه کاربر، درون حافظه خودش بنویسد که باعث خراب شدن داده‌های داخلی و حتی ایجاد راه برای اجرای کد مخرب با دسترسی کرنل خواهد شد.

پرسش 6: در صورتی که کاربر تغییراتی به صورت دستی در رجیسترهای بالا ایجاد کند، چه مشکلاتی به همراه خواهد داشت؟

در لینوکس آرگومانهای `syscall` در رجیسترهاست و برخی رجیسترها باید حفظ شوند. دستکاری دستی این رجیسترها قبل یا حین `syscall` باعث نقض `ABI` میشود و باعث میشود آرگومانهای اشتباه به کرنل برسد. خطاهایی مثل `EINVAL` و `EFAULT` برگردد یا روی منبع اشتباه عمل شود و برنامه هم به هم میریزد چون رجیسترهای `callee saved` مثل `edi esi ebx` و `ebp` دیگر مقدار درست ندارند و ممکن است کرش کند دستکاری `esp` یا `ebp` استک و فریم را خراب میکند و برگشت غیرممکن میشود. معمولاً رجیسترها را `save` و بعد `restore` میکند و تغییرات را نادیده میگیرد. در `xv6` آرگومانها از پشته برداشته میشوند اما باز هم دستکاری بی جا رجیسترها میتواند فریم و پرسها را خراب کند.

بهترین کار استفاده از استاب رسمی یا `inline asm` با اعلان دقیق ورودی خروجی و `clobber` است.

بررسی گام های اجرای فراخوانی سیستمی در سطح کرنل توسط gdb

یک برنامه سطح کاربر به نام pidtest نوشته شده و به Makefile اضافه شده که شماره پرده فعلی را با استفاده از سیستم کال getpid () چاپ می کند.

```
C: > Users > NoteBook > Desktop > xv6-public-master > xv6-public-master > C pidtest.c > ...
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(void){
7      int p = getpid();
8      printf(1, "pid=%d\n", p);
9      exit();
10 }
11 |
```

بعد از اتصال gdb به سیستم عامل یک breakpoint در خط 138 فایل syscall.c قرار می دهیم. با اجرای برنامه سطح کاربر، gdb در خط ذکر شده متوقف می شود و دستور bt را اجرا می کنیم . دستور bt در GDB برای نمایش پشته Backtrace استفاده میشود. این دستور تمامی فریمهای تابع در حال اجرا را لیست میکند، از جمله آدرسهای بازگشت، نام توابع، و محل مربوطه.

```
(gdb) bt
#0 syscall () at syscall.c:173
#1 0x80105ebd in trap (tf=0x8dffefb4) at trap.c:43
#2 0x80105c6a in alltraps () at trapasm.S:20
#3 0x8dffefb4 in ?? ()
Backtrace stopped: previous frame inner to this frame [corrupt]
```

0 Frame : -تابع فعلی که اجرا در آن متوقف شده syscall است.
1 Frame : -تابع trap با آرگومان tf صدا شده و اگر trapno برابر T_SYSCALL باشد syscall() را فراخوانی می کند. آدرس 0 x80105ebd محل همان call به syscall در خط 43 است.

2 Frame: -استاب اسمبلی ورود است که رجیسترها را پوش می‌کند، `trapframe` می‌سازد و `trap(tf)` را صدا می‌زند. پس از بازگشت از `trap` باقی کار را انجام می‌دهد تا در نهایت با `iret` به کاربر برگردد.

3 Frame: -این آدرس به فضای کاربر/ابتدای `trapframe` اشاره دارد و `gdb` نمادی برایش ندارد. همان عددی است که به عنوان `tf` دیده شد، بنابراین قابل دیباگ مثل فریم‌های C نیست.

-خط آخر: این پیام نشان می‌دهد که پشته ممکن است خراب شده باشد .

دستور `down` برای حرکت در فریم‌های پشته `Stack Frames` استفاده می‌شود. وقتی یک باگ یا توقف در تابعی رخ می‌دهد، `GDB` اطلاعات مربوط به فریم فعلی و فریم‌های بالا را در پشته نشان می‌دهد. وقتی در فریم‌های بالاتر پشته (مثل تابع فراخواننده) هستیم و می‌خواهیم به فریم پایین تر (توابع فرزند یا توابعی که مستقیم اجرا شده اند) حرکت کنیم، از `down` استفاده می‌کنیم. هر بار اجرای دستور ما را یک فریم پایین تر می‌برد.

```
(gdb) up
#1 0x80105ebd in trap (tf=0x8dffefb4) at trap.c:43
43 syscall();
(gdb) p tf->eax
$1 = 50
```

در اینجا با دستور `up` به فریم ۱ می‌رویم. مقدار رجیستر `eax` برابر ۵ می‌باشد. اما شماره فراخوان سیستمی `getpid()` برابر ۱۱ می‌باشد که برابر نمی‌باشند. دلیل آن است که برای اجرای یک برنامه سطح کاربر تعدادی فراخوانی سیستمی قبل و بعد آن اجرا می‌شوند. مثلاً برای اجرای کامند ورودی، پردازش `sh` موظف است تا با فراخوان سیستمی `fork` یک پردازش جدید بسازد و سپس خود پردازش جدید با فراخوان سیستمی `exec` برنامه را لود می‌کند و در اتمام کارش با فراخوان سیستمی `exit` به کار خودش پایان می‌دهد. همچنین پس از اتمام اجرای پردازش جدید، دوباره پردازش `sh` با فراخوان سیستمی علامت `$` را در صفحه چاپ کرده و با یک فراخوان سیستمی دیگر منتظر کامند جدید می‌ماند.

```
(gdb) p tf->eax
$7 = 11
```

همانطور که مشاهده می‌کنید مقدار رجیستر `eax` در چاپ هفتم برابر با شماره فراخوان سیستمی `getpid` می‌باشد.

ارسال آرگومان های فراخوانی های سیستمی

```
33  int
34  ✓ sys_simple_arithmetic_syscall(void)
35  {
36      struct proc *p = myproc();
37
38      int a = p->tf->ebx;
39      int b = p->tf->ecx;
40
41      int result = (a + b) * (a - b);
42
43      cprintf("Calc: (%d+%d)*(%d-%d) = %d\n", a, b, a, b, result);
44
45      return result;
46  }
```

در `sysproc.c` پیاده‌سازی تابع `sys_simple_arithmetic_syscall` در سطح کرنل نمایش داده شده است. در این تابع ابتدا با `myproc()` ساختار فرایند جاری دریافت می‌شود و سپس از درون آن مقادیر رجیسترهای `ecx` و `ebx` که شامل آرگومان‌های ورودی هستند خوانده می‌شوند. عملیات $(a + b) * (a - b)$ انجام می‌شود و نتیجه در متغیر `result` ذخیره می‌شود. با استفاده از `cprintf` مقادیر ورودی و خروجی در کرنل چاپ شده و در نهایت مقدار محاسبه‌شده به عنوان خروجی به برنامه کاربر بازگردانده می‌شود.

```
37  .globl simple_arithmetic_syscall
38  simple_arithmetic_syscall:
39      pushl %ebx
40      movl 8(%esp), %ebx
41      movl 12(%esp), %ecx
42      movl $SYS_simple_arithmetic_syscall, %eax
43      int $T_SYSCALL
44      popl %ebx
45      ret
46
```

در `usys.s` ابتدا مقدار رجیستر `ebx` برای حفظ مقدار اصلی در پشته ذخیره می‌شود. سپس دو آرگومان ورودی از روی پشته کاربر خوانده شده و به ترتیب در رجیسترهای `ecx` و `ebx` قرار داده می‌شوند. پس از آن شماره سیستم‌کال در `eax` قرار می‌گیرد و با دستور `int $T_SYSCALL` کنترل به هسته منتقل می‌شود. در پایان مقدار `ebx` از پشته بازیابی شده و تابع با دستور `ret` خاتمه می‌یابد. این قسمت رابط بین برنامه کاربر و سیستم‌کال در هسته را تشکیل می‌دهد.

```

C calc.c > ...
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      int a = 5, b = 3;
9      int r = simple_arithmetic_syscall(a, b);
10     printf(1, "user: result = %d\n", r);
11     exit();
12 }
13

```

در `calc.c` تابع `simple_arithmetic_syscall` با دو آرگومان عددی `a=5` و `b=3` فراخوانی می‌شود. این تابع همان سیستم‌کالی است که در هسته پیاده‌سازی کرده‌ایم و وظیفه دارد محاسبه‌ای ساده را انجام دهد و نتیجه را به برنامه کاربر بازگرداند. در پایان برنامه مقدار بازگشتی در متغیر `r` ذخیره شده و با `printf` چاپ می‌شود. این بخش در واقع ورودی داده‌ها را از سطح کاربر گرفته و آغازگر مسیر سیستم‌کال است.

```

$ calc
Calc: (5+3)*(5-3) = 16
user: result = 16
$ |

```

نحوه اضافه کردن فراخوانیهای سیستمی

برای اضافه کردن فراخوان سیستمی به سیستم عامل، باید چند جا آنها را اضافه کنیم.

usys.s

```
32 SYSCALL(make_duplicate)
33 SYSCALL(show_process_family)
34 SYSCALL(grep_syscall)
35 SYSCALL(set_priority_syscall)
36
37 .globl simple_arithmetic_syscall
38 simple_arithmetic_syscall:
39     pushl %ebx
```

Syscall.c

```
134 [SYS_simple_arithmetic_syscall] sys_simple_arithmetic_syscall,
135 [SYS_make_duplicate] sys_make_duplicate,
136 [SYS_show_process_family] sys_show_process_family,
137 [SYS_grep_syscall] sys_grep_syscall,
138 [SYS_set_priority_syscall] sys_set_priority_syscall,
139 };
```

Syscall.h

```
22 #define SYS_close 21
23 #define SYS_simple_arithmetic_syscall 22
24 #define SYS_make_duplicate 23
25 #define SYS_show_process_family 24
26 #define SYS_grep_syscall 25
27 #define SYS_set_priority_syscall 26
28
```

User.h

```
26 int simple_arithmetic_syscall(int a, int b);
27 int make_duplicate(const char *src_file);
28 int show_process_family(int pid);
29 int grep_syscall(const char* keyword, const char* filename, char* user_buffer, int buffer_size);
30 int set_priority_syscall(int pid, int priority);
31
32
```


1. پیاده سازی فراخوانی سیستمی ایجاد نسخه کپی فایل

در Sysfile.c در تابع `sys_make_duplicate` ابتدا رشته مسیر مبدأ با `argstr` بازیابی و اعتبارسنجی می‌شود؛ اگر شکست بخورد مقدار ۱ برمی‌گردانیم (خطای عمومی). سپس طول مسیر بررسی می‌شود تا بتوانیم «_copy» را بدون عبور از `MAXPATH` به انتهای آن اضافه کنیم. با دو `memmove` نام مقصد ساخته می‌شود: ابتدا کپی کامل `src` در `dst` و سپس الصاق «_copy» سپس با `begin_op` یک تراکنش فایل سیستمی آغاز می‌کنیم تا عملیات نام‌گذاری/ایجاد فایل از نظر ثبات روی دیسک اتمیک باشد `namei`. برای پیدا کردن `inode` مبدأ صدا زده می‌شود؛ اگر فایل نبود، با `end_op` خاتمه می‌دهیم و -۱ باز می‌گردانیم (مطابق نیازمندی «مبدأ وجود نداشت»). بعد با `ilock` نوع `inode` را می‌خوانیم و فقط اگر `T_FILE` بود ادامه می‌دهیم؛ در غیر این صورت قفل را آزاد و تراکنش را می‌بندیم و ۱ (خطا) برمی‌گردانیم. این بخش دقیقاً شرط‌های «ورودی معتبر و وجود فایل مبدأ» را پوشش می‌دهد و از همان الگوی خواندن امن آرگومان (`argstr`) که در پروژه تأکید شده پیروی می‌کند.

برای خواندن و نوشتن سطح کرنل از ساختار `file` استفاده می‌کنیم. ابتدا `f_src = filealloc` و مشخصه‌هایش تنظیم می‌شود تا `inode` مبدأ فقط-خواندنی شود؛ سپس با `create(dst, T_FILE, ...)` فایل مقصد ساخته می‌شود. اگر ساخت مقصد شکست بخورد، `f_src` بسته می‌شود و تراکنش پایان می‌یابد. بعد `f_dst = filealloc` گرفته و به `inode` مقصد وصل می‌شود و دسترسی‌هایش فقط-نوشتن تنظیم می‌گردد. سپس در یک حلقه با اندازه بافر ۵۱۲ بایت، `fileread` از `f_src` انجام می‌دهیم و همان مقدار را با `filewrite` در `f_dst` می‌نویسیم؛ اگر تعداد بایت نوشته‌شده با خوانده‌شده برابر نبود، `ret=1` و خروج از حلقه. در پایان خطای خواندن (`n<0`) بررسی، هر دو `file` بسته، `end_op` فراخوانی و مقدار `ret` بازگردانده می‌شود. این طراحی دقیقاً «کپی کامل در هسته» را پیاده می‌کند، از تراکنش `begin_op/end_op` برای سازگاری فایل سیستم استفاده می‌کند و مسیرهای خطا را پوشش می‌دهد تا مطابق قرارداد خروجی، ۱/-۱ برگردد.

```

561 int
562 sys_make_duplicate(void)
563 {
564     char *src;
565     char dst[MAXPATH];
566     int len;
567     struct inode *ip_src, *ip_dst;
568     struct file *f_src = 0, *f_dst = 0;
569     char buf[512];
570     int n;
571     int ret = 0;
572
573     if(argstr(0, &src) < 0)
574         return 1;
575
576     len = strlen(src);
577     if(len + 6 > MAXPATH)
578         return 1;
579
580     memmove(dst, src, len);
581     memmove(dst + len, "_copy", 6);
582
583     begin_op();
584
585     if((ip_src = namei(src)) == 0){
586         end_op();
587         return -1;
588     }
589
590     ilock(ip_src);
591     if(ip_src->type != T_FILE){
592         iunlockput(ip_src);
593         end_op();
594         return 1;
595     }
596
597     if((f_src = filealloc()) == 0){
598         iunlockput(ip_src);
599         end_op();
600         return 1;
601     }
602

```

```
603 f_src->type = FD_INODE;
604 f_src->ip = ip_src;
605 f_src->off = 0;
606 f_src->readable = 1;
607 f_src->writable = 0;
608 iunlock(ip_src);
609
610 if((ip_dst = create(dst, T_FILE, 0, 0)) == 0){
611     fclose(f_src);
612     end_op();
613     return 1;
614 }
615
616 if((f_dst = filealloc()) == 0){
617     iunlockput(ip_dst);
618     fclose(f_src);
619     end_op();
620     return 1;
621 }
622
623 f_dst->type = FD_INODE;
624 f_dst->ip = ip_dst;
625 f_dst->off = 0;
626 f_dst->readable = 0;
627 f_dst->writable = 1;
628 iunlock(ip_dst);
629
630 while((n = fileread(f_src, buf, sizeof(buf))) > 0){
631     if(filewrite(f_dst, buf, n) != n){
632         ret = 1;
633         break;
634     }
635 }
636
637 if(n < 0)
638     ret = 1;
639
640 fclose(f_src);
641 fclose(f_dst);
642 end_op();
643
644 return ret;
645 }
```

برنامه کاربر

duplicate.c

این برنامه فقط نقش رابط بین کاربر و هسته را دارد و خودش هیچ عملیات کپی روی فایل انجام نمی‌دهد. کاربر هنگام اجرا باید نام فایل منبع را به‌عنوان آرگومان وارد کند و اگر تعداد آرگومان‌ها اشتباه باشد پیام "usage: dup file" چاپ می‌شود. سپس با فراخوانی `make_duplicate(argv[1])` عملیات کپی در سطح کرنل انجام می‌شود. نتیجه‌ی بازگشتی بررسی می‌شود؛ اگر مقدار صفر باشد یعنی کپی موفق بوده و پیام "duplicate ok" نمایش داده می‌شود، اگر مقدار منفی یک باشد یعنی فایل منبع پیدا نشده و در بقیه موارد پیام "duplicate failed" چاپ می‌شود. در این برنامه تمام کار واقعی در هسته انجام می‌شود و بخش کاربر فقط ورودی می‌گیرد و خروجی را نمایش می‌دهد.

```
C duplicate.c > ...
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      if(argc != 2){
9          printf(2, "usage: dup file\n");
10         exit();
11     }
12
13     int r = make_duplicate(argv[1]);
14
15     if(r == 0)
16         printf(1, "duplicate ok\n");
17     else if(r == -1)
18         printf(1, "source file not found\n");
19     else
20         printf(1, "duplicate failed\n");
21
22     exit();
23 }
24
```

```
$ echo hello > a
$ duplicate a
duplicate ok
$ cat a_copy
hello
$ |
```

2. پیاده سازی فراخوانی سیستمی بدست آوردن اعضای خانواده یک پردازش

در `proc.c` تابع `show_process_family` با گرفتن `pid` آغاز می‌شود. ابتدا قفل جدول پردازش‌ها با `acquire(&ptable.lock)` گرفته می‌شود تا پیمایش‌ها سازگار باشند. سپس یک گذر روی `ptable` انجام می‌شود تا پردازش‌های هدف پیدا شود؛ اگر پیدا نشد، قفل آزاد و 1- برگردانده می‌شود. در ادامه اگر پردازش والد داشته باشد، خط `"My id: X, My parent id: Y"` چاپ می‌شود و اگر نداشت، `Y` را 1- چاپ می‌کند. برای فرزندان دوباره روی `ptable` پیمایش می‌شود و هر پردازش‌ای که `parent` آن برابر `target` باشد با `"Child pid: ..."` چاپ می‌گردد؛ اگر هیچ فرزندی پیدا نشود، پیام `"No children."` چاپ می‌شود. برای برادرها، اگر `target` والد دارد، دوباره پیمایش می‌شود و هر پردازش‌ای که `parent` آن همان `parent target` است و خودش `target` نیست به عنوان `sibling` چاپ می‌گردد؛ اگر چیزی پیدا نشود `"No siblings."` چاپ می‌شود. در پایان قفل با `release` آزاد شده و مقدار 0 برگردانده می‌شود. در تمام این مسیر از فیلدهای موجود در `struct proc` استفاده شده و تغییری در ساختار داده لازم نیست، فقط رعایت قفل‌گیری برای همگامی الزامی است.

```

int
show_process_family(int pid)
{
    struct proc *p;
    struct proc *target = 0;
    int have_child = 0;
    int have_sibling = 0;

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state != UNUSED && p->pid == pid){
            target = p;
            break;
        }
    }

    if(target == 0){
        release(&ptable.lock);
        return -1;
    }

    if(target->parent)
        cprintf("My id: %d, My parent id: %d\n", target->pid, target->parent->pid);
    else
        cprintf("My id: %d, My parent id: -1\n", target->pid);

    cprintf("Children of process %d:\n", target->pid);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state != UNUSED && p->parent == target){
            cprintf("Child pid: %d\n", p->pid);
            have_child = 1;
        }
    }
    if(!have_child)
        cprintf("No children.\n");

    cprintf("Siblings of process %d:\n", target->pid);
    if(target->parent){
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != UNUSED && p->parent == target->parent && p != target){
                cprintf("Sibling pid: %d\n", p->pid);
                have_sibling = 1;
            }
        }
    }
}

```

```

69     if(!have_sibling)
70         cprintf("No siblings.\n");
71
72     release(&ptable.lock);
73     return 0;
74 }
75
76 void
77 pinit(void)
78 {
79     initlock(&ptable.lock, "ptable");
80 }
81

```

تابع سیستم‌کال sys_show_process_family در sysproc.c

این تابع آرگومان pid را با argint دریافت می‌کند. اگر دریافت آرگومان نامعتبر بود 1- برمی‌گرداند، در غیر این صورت مستقیماً show_process_family(pid) را صدا می‌زند و مقدار بازگشتی آن را به فراخوان درخواست‌کننده برمی‌گرداند.

```

25 int
26 sys_show_process_family(void)
27 {
28     int pid;
29     if(argint(0, &pid) < 0)
30         return -1;
31     return show_process_family(pid);
32 }

```

Find_family.c

برنامه‌ی سطح کاربر یک آرگومان می‌گیرد؛ اگر تعداد آرگومان‌ها نادرست باشد، "usage: fam pid" را چاپ می‌کند و خارج می‌شود. سپس pid از argv[1] با atoi خوانده می‌شود و show_process_family(pid) فراخوانی می‌شود. پس از اتمام، مقدار بازگشتی r چاپ می‌گردد تا بتوان نتیجه‌ی موفقیت یا خطا را مشاهده کرد. این برنامه فقط نقش رابط دارد و هیچ پیمایشی روی جدول پدازه‌ها در سطح کاربر انجام نمی‌دهد.

C find_family.c > ...

```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      if(argc != 2){
9          printf(1, "usage: fam pid\n");
10         exit();
11     }
12     int pid = atoi(argv[1]);
13     int r = show_process_family(pid);
14     printf(1, "ret = %d\n", r);
15     exit();
16 }
```

```
$ find_family 2
My id: 2, My parent id: 1
Children of process 2:
Child pid: 11
Siblings of process 2:
No siblings.
ret = 0
$ |
```


3. پیاده سازی فراخوانی سیستمی جستجوی محتوا در فایل

در Sysfile.c

در ابتدای `sys_grep_syscall` آرگومان‌ها به ترتیب با `argstr` برای `keyword` و `filename`، با `argptr` برای `user_buffer` و با `argint` برای `bufsize` بازیابی و اعتبارسنجی می‌شوند. اگر بازیابی هر کدام نامعتبر باشد یا `bufsize ≤ 1`، تابع با خطا برمی‌گردد. سپس یک بافر موقت هسته‌ای با `kalloc` گرفته می‌شود تا محتوا بخش‌بخش خوانده شود. برای دسترسی به فایل `begin_op` فراخوانی می‌شود، با `namei` نام فایل به `inode` تبدیل می‌گردد و در صورت نبود فایل، `end_op` اجرا و خطا بازگردانده می‌شود. در ادامه با `ilock` قفل `inode` گرفته می‌شود. حلقه خواندن با `readi(ip, chunk, off, PGSIZE)` انجام می‌شود؛ هر بار تا یک صفحه از فایل در `chunk` خوانده می‌شود و `off` به اندازه `n` افزایش می‌یابد.

پس از هر بار خواندن، بایت‌های `chunk` اسکن می‌شوند و کاراکترها در `line_buf` انباشته می‌گردند تا به `'\n'` برسیم. با رسیدن به انتهای یک خط، تابع `contains_keyword(line_buf, line_len, keyword)` بررسی می‌کند آیا کلیدواژه در این خط وجود دارد یا نه. در صورت وجود، طول قابل کپی شدن محاسبه می‌شود: `copy_len = min(line_len, bufsize-1)` تا همیشه فضای کافی برای نال تمام‌کننده باقی بماند. سپس با `copyout(myproc()->pgdir, (uint)user_buf, line_buf, copy_len)` محتوی خط به بافر کاربر کپی می‌شود و با یک `copyout` دیگر کاراکتر صفر انتهایی در `user_buf+copy_len` نوشته می‌شود. اگر `copyout` شکست بخورد خطا برگردانده می‌شود؛ در غیر این صورت وضعیت موفقیت ثبت می‌شود و جستجو متوقف می‌گردد. اگر کل `chunk` طی شد و هنوز خط کامل نشده بود، تجمع ادامه پیدا می‌کند تا خط بعدی کامل شود. در پایان، در صورت نبود هیچ تطابقی، وضعیت «عدم تطابق» گزارش می‌شود. پس از اتمام کار `iunlockput(ip)` برای آزادسازی `inode` و `end_op` برای بستن تراکنش فایل سیستمی فراخوانی می‌شود و `chunk` با `kfree` آزاد می‌گردد.

```

43 }
44 int
45 sys_grep_syscall(void)
46 {
47     char *keyword;
48     char *filename;
49     char *user_buf;
50     int bufsize;
51     struct inode *ip;
52     char *chunk;
53     int off = 0;
54     int n;
55     char line_buf[512];
56     int line_len = 0;
57     int found = 0;
58     int r = 1;
59
60     if(argstr(0, &keyword) < 0)
61         return -1;
62     if(argstr(1, &filename) < 0)
63         return -1;
64     if(argint(3, &bufsize) < 0)
65         return -1;
66     if(bufsize <= 1)
67         return -1;
68     if(argptr(2, &user_buf, bufsize) < 0)
69         return -1;
70
71     chunk = (char*)kalloc();
72     if(chunk == 0)
73         return -1;
74
75     begin_op();
76     ip = namei(filename);
77     if(ip == 0){
78         end_op();
79         kfree(chunk);
80         return -1;
81     }
82
83     ilock(ip);
84
85     for(;;){
86         n = readi(ip, chunk, off, PGSIZE);
87         if(n <= 0)
88             break;
89         off += n;
90

```

```

45 sys_grep_syscall(void)
85     for(;;){
90
91         int i = 0;
92         while(i < n && !found){
93             char c = chunk[i++];
94
95             if(c == '\n'){
96                 if(contains_keyword(line_buf, line_len, keyword)){
97                     int copy_len = line_len;
98                     if(copy_len > bufsize - 1)
99                         copy_len = bufsize - 1;
100                     if(copyout(myproc()->pgdir, (uint)user_buf, line_buf, copy_len) < 0){
101                         r = -1;
102                     } else {
103                         char zero = 0;
104                         if(copyout(myproc()->pgdir, (uint)(user_buf + copy_len), &zero, 1) < 0)
105                             r = -1;
106                         else {
107                             r = 0;
108                         }
109                     }
110                     found = 1;
111                     break;
112                 }
113                 line_len = 0;
114             } else {
115                 if(line_len < (int)sizeof(line_buf) - 1){
116                     line_buf[line_len++] = c;
117                 }
118             }
119         }
120
121         if(found)
122             break;
123     }
124
125     if(!found && r == 1)
126         r = 1;
127
128     iunlockput(ip);
129     end_op();
130     kfree(chunk);
131
132     return r;
133 }

```

برنامه **grep_test.c** سه آرگومان می‌گیرد: کلیدواژه، نام فایل و بافر محلی 256 بایتی برای دریافت خروجی. ابتدا استفاده صحیح را بررسی می‌کند و سپس `grep_syscall(argv[1], argv[2], buf, sizeof(buf))` را صدا می‌زند. براساس مقدار بازگشتی پیام مناسب چاپ می‌شود؛ در کد فعلی اگر `ret==0` پیام `found` و اگر `ret==1` پیام `no match` و در غیر این صورت `error` چاپ می‌شود.

C grep_test.c > ...

```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      if(argc != 3){
9          printf(1, "usage: grep_test keyword filename\n");
10         exit();
11     }
12
13     char buf[256];
14     int ret = grep_syscall(argv[1], argv[2], buf, sizeof(buf));
15
16     if(ret == 0)
17         printf(1, "found: %s\n", buf);
18     else if(ret == 1)
19         printf(1, "no match\n");
20     else
21         printf(1, "error\n");
22
23     exit();
24 }
25
```

```
init: starting sh
$ echo hello world > a
$ echo hello world > b
$ grep_test hello a
found: hello world
$ grep_test hello b
found: hello world
$ grep_test xyz README
no match
$ |
```

4. پیاده سازی فراخوانی سیستمی تنظیم اولویت

در proc.c

تابع `set_priority(pid, priority)` ابتدا محدوده‌ی ورودی را بررسی می‌کند و اگر `priority` خارج از `{0,1,2}` باشد -1 برمی‌گرداند. سپس با `acquire(&ptable.lock)` قفل جدول پردازها گرفته می‌شود و روی `ptable` پیمایش انجام می‌گیرد. وقتی پردازهای با `pid` داده‌شده و `state != UNUSED` پیدا شد،

فیلد `p->priority` همان‌جا به مقدار جدید تنظیم می‌شود، قفل آزاد و 0 بازگردانده می‌شود. اگر `pid` یافت نشود، پس از آزاد کردن قفل مقدار -1 برمی‌گردد. این بخش مسئول «ذخیره‌سازی وضعیت اولویت» داخل کرنل است.

```
369     }
370     int
371     set_priority(int pid, int priority)
372     {
373         struct proc *p;
374
375         if(priority < 0 || priority > 2)
376             return -1;
377
378         acquire(&ptable.lock);
379         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
380             if(p->pid == pid && p->state != UNUSED){
381                 p->priority = priority;
382                 release(&ptable.lock);
383                 return 0;
384             }
385     }
386     release(&ptable.lock);
387     return -1;
388 }
389
390
```

در Sysproc.c

ابع `sys_set_priority_syscall` با `argint` دو آرگومان `pid` و `prio` را از فضای کاربر می‌گیرد. در صورت نامعتبر بودن هر کدام، -1 بازمی‌گرداند. در غیر این صورت مستقیماً `set_priority(pid, prio)` را فراخوانی و مقدار بازگشتی آن را به کاربر برمی‌گرداند. این تابع حلقه‌ی اتصال مسیر `sys_*` → `syscall` → `trap` با منطق اصلی در `proc.c` است.

```
12  int
13  sys_set_priority_syscall(void)
14  {
15      int pid;
16      int prio;
17
18      if(argint(0, &pid) < 0)
19          return -1;
20      if(argint(1, &prio) < 0)
21          return -1;
22
23      return set_priority(pid, prio);
24  }
25  int
```

Priority_test.c

برنامه دو فرزند CPU-bound ایجاد می‌کند که با تابع `busy` بار محاسباتی سنگین تولید می‌کنند. فرزند اول بلافاصله پس از `fork` اولویتش روی 0 (بالا) و فرزند دوم روی 2 (پایین) تنظیم می‌شود. هر فرزند در حلقه کار کرده و کاراکتر مربوطه را چاپ می‌کند. والد دو بار `wait` می‌زند و در پایان یک خط خالی چاپ می‌کند. انتظار رفتاری این است که با زمان‌بند اصلاح‌شده، خروجی فرزند با اولویت 0 زودتر و متراکم‌تر دیده شود و معمولاً زودتر تمام کند.

